

Beyond the Syllabus

How Fast Can Matching Go? Continuous Methods, Combinatorial Counterattacks, and
Online Arrivals

Week 11 Supplement – TOAE209/TOAH209: Design and Analysis of Algorithms

Foreign Trade University

Outline

Why look beyond the syllabus?

- This week you reduced **maximum bipartite matching** to a max-flow computation: build $s \rightarrow X \rightarrow Y \rightarrow t$ with unit capacities, run Edmonds–Karp, read off the matching. Since Edmonds–Karp augments along shortest paths, this reduction runs in $O(VE)$ – and for bipartite graphs specifically, the classic Hopcroft–Karp algorithm (a smarter augmenting-path schedule) gets this down to $O(E\sqrt{V})$, a bound that stood as the best known for almost **fifty years** (1973–2020).
- Three questions a curious student should ask next:
 - ① Can matching beat $O(E\sqrt{V})$ at all – and if so, how?
 - ② Does beating it *require* heavy continuous-optimization machinery, or can pure combinatorics get there too?
 - ③ Everything above assumes the whole graph is known upfront. What if vertices of Y (jobs, ads, riders) **arrive online**, one at a time, and you must match them irrevocably as they come?

How this supplement is different from a survey

Every section is grounded in a specific, verifiable paper (with arXiv/DOI); none of the bounds below are approximate folklore.

Fifty years of $O(E\sqrt{V})$

- Hopcroft & Karp (1973) showed that after $O(\sqrt{V})$ phases of *shortest-augmenting-path* search, a bipartite matching is maximum; each phase costs $O(E)$, giving $O(E\sqrt{V})$ total.
- Dinic (1970) and Karzanov (1973) proved essentially the same bound independently, via blocking flows – three different derivations of the same wall.
- For dense graphs ($E = \Theta(V^2)$), $O(E\sqrt{V}) = O(V^{2.5})$: at $V = 10^5$, that is $\sim 3 \times 10^{12}$ operations – far too slow.

The break: continuous optimization enters combinatorics

van den Brand, Lee, Nanongkai, Peng, Saranurak, Sidford, Song & Wang (FOCS 2020)

“Bipartite Matching in Nearly-linear Time on Moderately Dense Graphs.” arXiv:2009.01802.

A randomized algorithm solving maximum bipartite matching (and several relatives: transshipment, negative-weight shortest paths, optimal transport) in $\tilde{O}(m + n^{1.5})$ time.

- For moderately dense graphs ($m = \Omega(n^{1.5})$), this is **nearly linear in the input size** – the first improvement over Dinic/Hopcroft–Karp/Karzanov’s $O(m\sqrt{n})$ in 47 years.
- **How:** recast matching as a linear program, solve it with an **interior-point method** (IPM). Each IPM iteration reduces to solving a **Laplacian linear system** on the graph – and those have had nearly-linear-time solvers since Spielman & Teng (2004). Dynamic graph data structures avoid recomputing each Laplacian solve from scratch as the IPM progresses.

Putting numbers on it

At $n = 10^6$, $m = n^{1.5} \approx 10^9$:

Algorithm	Bound	Rough op. count
Hopcroft–Karp / Dinic (1970s)	$O(m\sqrt{n})$	$\sim 10^{12}$
van den Brand et al. (FOCS 2020)	$\tilde{O}(m + n^{1.5})$	$\sim 10^9$

Three orders of magnitude, from a fifty-year-old wall, by importing a tool (Laplacian solvers) that this week's lecture never mentions – matching is not just a graph problem once you let continuous optimization in.

A combinatorial counterattack

Chuzhoy & Khanna (SODA 2024 / STOC 2024)

“A Faster Combinatorial Algorithm for Maximum Bipartite Matching” (arXiv:2312.12584, SODA 2024, $\tilde{O}(m^{1/3}n^{5/3})$ -time) and “Maximum Bipartite Matching in $n^{2+o(1)}$ Time via a Combinatorial Algorithm” (arXiv:2405.20861, STOC 2024).

- Both results match or beat the IPM-based bound above – using **no** interior-point methods, **no** Laplacian solvers, only combinatorial augmenting-path ideas pushed further than Hopcroft–Karp’s.
- The lesson: the 2020 breakthrough proved *that* $O(m\sqrt{n})$ could be beaten, but it did **not** prove that continuous machinery was *necessary*. That is a separate, harder question – and as of 2024 the answer leans “no, elbow grease still works too.”

Why this matters pedagogically

Two independent, correct routes to the same destination is exactly the kind of thing worth noticing: a faster *bound* does not tell you the *only* way to achieve it.

The online model: Karp–Vazirani–Vazirani (1990)

- Everything above assumes X , Y , and all edges are known before you start. In **online bipartite matching**, X is fixed but vertices of Y arrive one at a time, each revealing its edges to X on arrival; you must match it immediately (or leave it unmatched) with **no take-backs**.
- Karp, Vazirani & Vazirani (STOC 1990) proved the RANKING algorithm – pick one random permutation of X up front, always match an arriving y to its *earliest-ranked* available neighbor – achieves competitive ratio $1 - 1/e \approx 0.632$ against the offline optimum, and that no online algorithm can beat $1 - 1/e$ in the worst case. A genuinely tight bound, open for “how do we do better” ever since – *if* the adversary is fully unconstrained.

Beating $1 - 1/e$ with (untrusted) predictions

Choo, Gouleakis, Ling & Bhattacharyya (ICML 2024)

“Online bipartite matching with imperfect advice.” PMLR 235:8762–8781, arXiv:2405.09784.

- Setting: before the online phase, the algorithm is handed a **prediction** of the arrival sequence – possibly wrong, possibly adversarially wrong.
- Idea: use a **prefix** of the actual arrivals as a statistical test of whether the prediction looks trustworthy; if it does, follow it (and beat $1 - 1/e$ when it is accurate); if not, fall back to a RANKING-style worst-case-safe baseline.
- This is the **algorithms-with-predictions** paradigm (Mitzenmacher & Vassilvitskii): a *consistency* guarantee (near-optimal if predictions are good) traded against a *robustness* guarantee (never worse than $1 - 1/e$ -ish even if predictions are adversarial) – both guarantees proven, not just empirically observed.

Summary

- 1 **The 1973 barrier fell in 2020:** van den Brand et al. (FOCS 2020) solve bipartite matching in $\tilde{O}(m + n^{1.5})$ by routing through Laplacian solvers and interior-point methods – continuous mathematics inside a purely combinatorial problem.
- 2 **Machinery is not mandatory:** Chuzhoy & Khanna (SODA/STOC 2024) reach a comparable bound with a purely combinatorial algorithm – there is more than one road to a faster algorithm.
- 3 **“Known upfront” is a strong assumption:** once arrivals are online, even the optimal worst-case algorithm (Karp–Vazirani–Vazirani, 1990) is stuck at $1 - 1/e$ – and only a genuinely new resource, untrusted *predictions* (Choo et al., ICML 2024), lets you provably beat that wall while staying safe when the predictions are wrong.

Looking ahead

Next week (Kleinberg & Tardos, Ch. 8) asks *which problems provably have no polynomial-time exact algorithm* – NP-completeness. “Optimal Account Balancing,” this week’s Hard problem, is already a preview: minimizing the transaction *count* has no known polynomial algorithm.